PROYECTO SELLO · UNIDAD II · SUSTENTACIÓN S12

CASAS

CASA O NADA

Plataforma inmobiliaria distribuida

Compra y alquiler, de la búsqueda al contrato: seguro, resiliente, consistente y observable.



Jannys Graciela Navarro Acrota

Miguel Angel Alanguia Japura

Grupo 9

ALCANCE · 2–3 MIN · GRUPAL

Jannys

Compra y alquiler de inmuebles, de la búsqueda al contrato

C Cliente

Se registra, explora y decide

- Filtra y guarda favoritos
- Pide una visita (opcional)
- Reserva 24 h y paga con captura
- Acepta su contrato en línea

A Agente

Publica y verifica

- Publica y edita sus inmuebles
- Confirma y reprograma visitas
- Revisa la captura: aprueba, observa o rechaza
- Envía y activa el contrato

A Administrador

Supervisa todo

- Indicadores y operaciones
- Crea agentes con acceso
- Activa o desactiva usuarios
- Revisa el sistema (Grafana, Eureka)

Explorar → **Reservar · 24 h** → **Pagar con captura** → **Verifica el agente** → **Contrato**

Dominio propio (inmobiliario), con nueve servicios independientes. El pago es manual y no mueve dinero real.

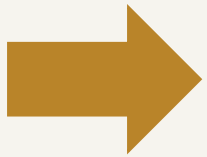
Los dos

De CRUD aislado a un sistema seguro, resiliente y observable

UNIDAD I

6 dominios de negocio

- Propiedad, agente, visita, reserva, pago y contrato
- CRUD y persistencia por dominio
- Config Server, Eureka y Gateway
- Una API de negocio por servicio



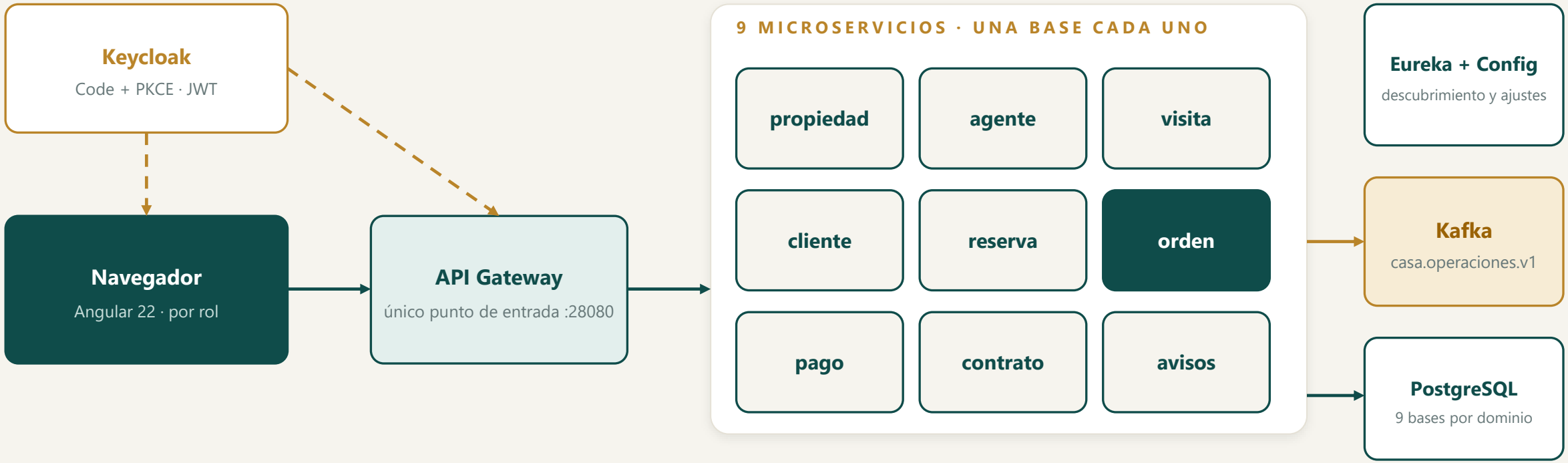
UNIDAD II

9 dominios (+ cliente, orden y avisos)

- Feign con timeout y Circuit Breaker
- Keycloak y permisos en los 9 servicios
- Kafka con outbox, inbox y DLT
- Saga orquestada con compensación
- Grafana, Loki y trace entre servicios
- Angular 22 con rutas por rol

Alanguia

Cliente → Gateway → microservicios → Keycloak y Kafka



Observabilidad: Prometheus (métricas) · Alloy + Loki (logs) · Grafana (panel) · X-Trace-ID entre servicios

Angular nunca conoce la dirección de una instancia: solo habla con el Gateway.

ASIGNACIÓN DE TAREAS · 2–3 MIN · GRUPAL

Los dos

Cada integrante es dueño de capacidades que se verifican en vivo

Jannys

Producto, seguridad por rol y panel

S11 **Frontend Angular + Keycloak**

Flujo completo con ruta protegida por rol

S7 **Seguridad por rol**

Acceso permitido y denegado (401 / 403)

S10 **Observabilidad**

Trace de la operación recién creada en Grafana

Alanguia

Resiliencia, eventos y consistencia

S6 **Resiliencia**

Caída real de propiedad y fallback

S8 **Mensajería**

Evento publicado y consumido en Kafka

S9 **Consistencia**

Compensación y reproceso idempotente

S7 **JWT en los 9 servicios**

Validación directa, sin pasar por el Gateway

DEMO · COBERTURA DE LA GUÍA DEL DOCENTE

Los dos

Los 8 pasos de la demo y quién muestra cada uno

#	Paso que pide el docente	Quién	Dónde se ve
1	Pitch ejecutivo: evolución desde la Unidad I	**Los dos**	Diapositivas 2 y 3
2	Caída provocada de una instancia y fallback	**Alanguia**	test-resilience.ps1
3	Seguridad: acceso permitido y denegado en más de un servicio	**Jannys · Alanguia**	Web por rol · test-distributed-security.ps1
4	Evento de negocio publicado y consumido	**Alanguia**	Kafka UI
5	Fallo a mitad del proceso: compensación o reproceso idempotente	**Alanguia**	Pago rechazado · test-replay.ps1
6	Panel de observabilidad con dos o más servicios	**Jannys**	Grafana (trace)
7	Flujo completo desde el frontend, con ruta protegida por rol	**Jannys**	Web :4300
8	Cierre con decisiones propias	**Los dos**	Última diapositiva

Cada uno: 3 minutos de demo y 3 de preguntas.

Alanguia

Si propiedad cae, reserva responde sin colgarse



Prueba: scripts/test-resilience.ps1 → PASS (CLOSED → OPEN → HALF_OPEN → CLOSED)

Estados del circuito



Lectura

Devuelve los datos locales de la reserva con degradado = true.

Escritura

Responde 503 y no inventa datos: crear una reserva exige validar el inmueble de verdad.

Los dos

Seguridad en tres capas, validada en cada microservicio

1 · Web (Angular)

Code + PKCE · token en memoria · rutas por rol

2 · API Gateway

valida el JWT y las rutas permitidas por rol

3 · Cada microservicio

valida firma y emisor (JWK) + pertenencia del dato

Por qué no basta el Gateway: alguien podría llamar al puerto interno de un servicio. Por eso los 9 servicios validan el token por su cuenta.

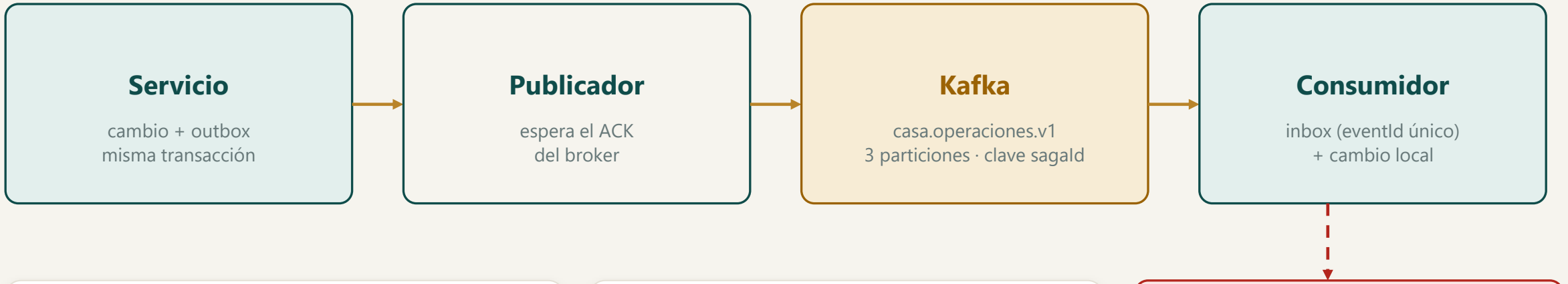
Lo que se comprueba en vivo

401	**Sin token**
403	**Cliente en una ruta de ADMIN**
200	**Cada rol en lo suyo**

El cliente solo ve lo suyo: su cliente_id viene del token firmado y no se puede cambiar desde el cuerpo de la petición.

Alanguia

Eventos con Kafka: desacoplados y sin perder mensajes



Lo que ganamos

- El productor sigue aunque el consumidor esté caído
- Los servicios no se conocen entre sí
- Un duplicado no repite el efecto

Lo que perdemos

- Inmediatez: hay estados intermedios visibles
- Hay que tratar duplicados
- Entrega «al menos una vez», no exactamente una

Demo: Kafka UI → casa.operaciones.v1 → Messages

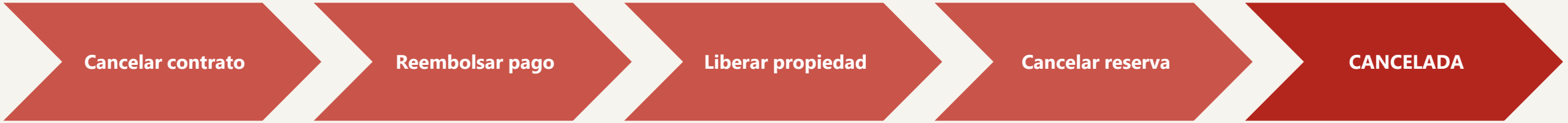
Alanguia

Una saga orquestada por orden-ms, con compensación en orden

CAMINO FELIZ



SI EL PAGO SE RECHAZA O VENCE EL PLAZO: COMPENSACIÓN



No hay ACID global

Cada base confirma su transacción local; la saga coordina con mensajes.

Idempotencia

Inbox por eventId y X-Idempotency-Key: repetir no duplica el efecto.

Decisión propia

Orquestada (el curso muestra coreografía): el estado y las compensaciones se ven en un solo lugar.

Jannys

Una operación, una traza: de la web a cada servicio



Grafana · panel CASAS · Operación distribuida

Métricas

Prometheus: servicios arriba, errores 5xx y latencia p95

Logs

Alloy los envía a Loki; se filtran por traceId

Trace

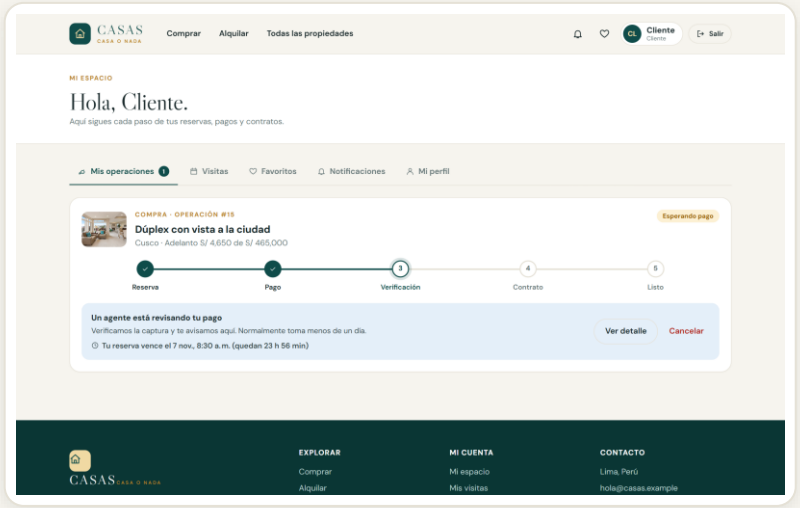
X-Trace-ID viaja por HTTP y por los eventos

Salud

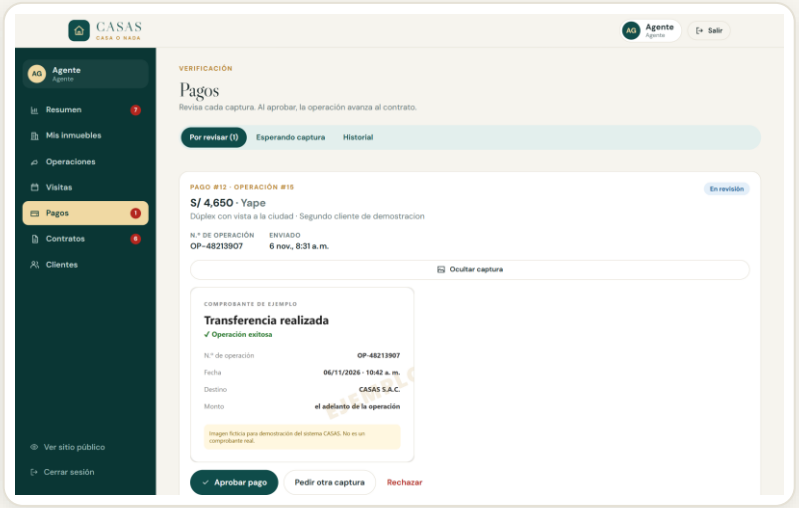
liveness (vive) y readiness (incluye la base de datos)

Jannys

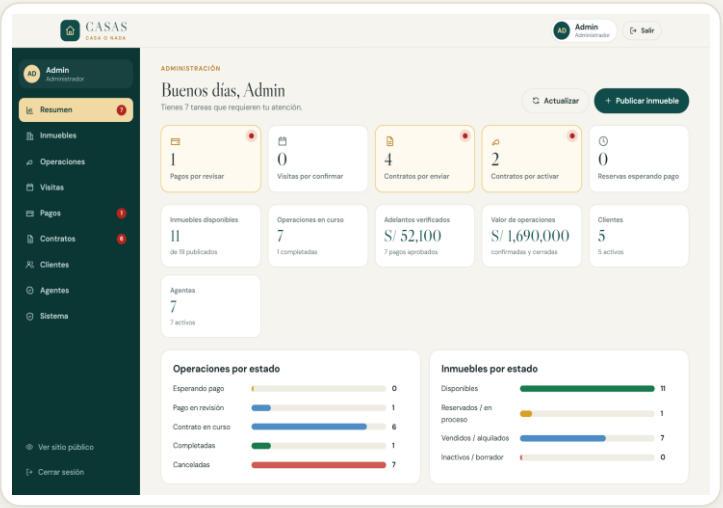
Angular 22 con Keycloak: cada rol ve lo suyo



Cliente · Mi espacio con el progreso de la operación



Agente · revisión del pago con la captura



Administrador · resumen e indicadores

Authorization Code + PKCE

Token en memoria; nunca en localStorage

Solo el Gateway

La web no conoce direcciones de instancias

Rutas por rol

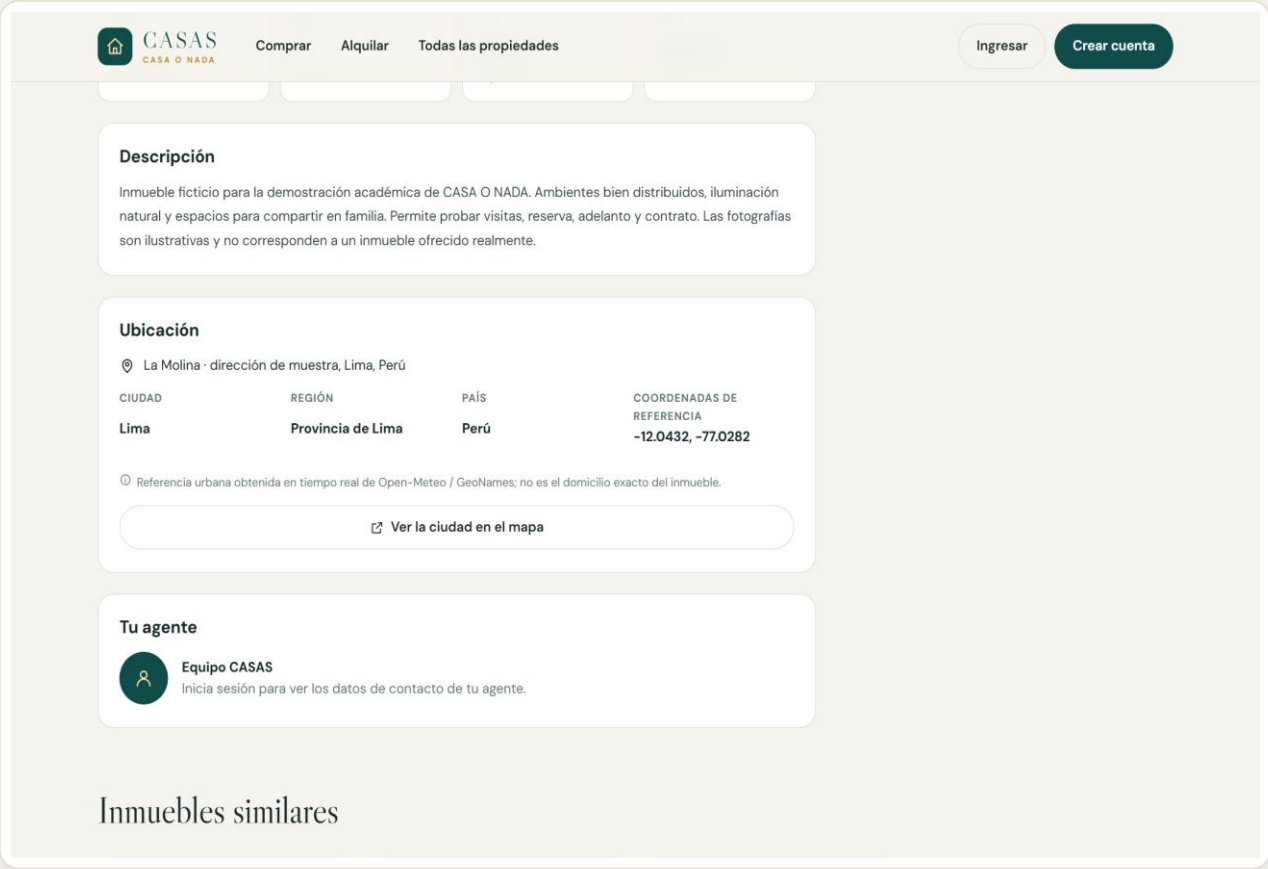
Cliente → Mi espacio · Agente y admin → Panel

Servidor manda

Ocultar un botón no da permisos; la API valida igual

Jannys

Un servicio externo real: la ubicación de la ciudad



Ficha del inmueble con datos reales de Open-Meteo / GeoNames

Angular
GET /propiedades/ubicaciones?ciudad=…

↓

Gateway
ruta pública de lectura

↓

propiedad-ms
UbicacionController

↓

Open-Meteo
geocoding-api.open-meteo.com

Timeout 4 s · caché 30 min · si falla, responde «no disponible» y todo sigue funcionando.

Los dos

Cómo cubrimos la rúbrica del docente

#	Criterio	Peso	Evidencia que mostramos
1	**Respuesta controlada ante fallos**	**12 %**	Caída real de propiedad: CLOSED → OPEN → HALF_OPEN → CLOSED
2	**Seguridad distribuida y rutas**	**16 %**	401 / 403 en los 9 servicios; rutas por rol en la web
3	**Mensajería con servicios desacoplados**	**16 %**	Evento publicado y consumido en Kafka UI
4	**Consistencia, compensación e idempotencia**	**16 %**	Pago rechazado → compensación; mismo evento → 1 contrato
5	**Logs, health, métricas y paneles**	**12 %**	Grafana: un trace por varios servicios
6	**Frontend integrado por el Gateway**	**8 %**	Flujo completo con ruta protegida por rol
7	**Sustentación**	**20 %**	Aporte individual, repositorio, MkDocs y pitch

Los dos

Probado de verdad y reproducible por otra persona

91	50	9 / 9	1
pruebas unitarias	**comprobaciones HTTP**	**servicios exigen JWT**	**contrato tras reenviar**
sin fallos	todas PASS	directo, sin Gateway	el mismo evento

Repositorio y documentación

- GitHub: Stephmarc/CASAS-distribuidas
- MkDocs: stephmarc.github.io/CASAS-distribuidas
- Guías de demo, evidencias y código en el repositorio

Para reproducirlo

Docker Desktop abierto y un solo comando:

```
.\scripts\start-complete.ps1
```

Compila, corre las 91 pruebas, levanta los servicios y abre la web.

Jannys

Las APIs que añadimos y dónde están en el código

API	Para qué	Dónde está en el código	Qué necesita
Open-Meteo / GeoNames	Ubicación real de la ciudad en la ficha	`propiedad-ms · UbicacionController`	**Sin clave**
RENIEC por DNI (apis.net.pe)	Autocompleta nombres al registrarse	`cliente-ms · RegistroController`	**Token gratis**
Keycloak Admin REST	Crea usuarios, asigna rol, activa o desactiva	`casa-common · KeycloakAdmin`	**Admin de Keycloak**
Correo (SMTP)	Bienvenida, reserva y acceso de agente	`casa-common · Mailer y EmailTemplates`	**Mailpit o Gmail**

Cómo están hechas: el servicio llama a la API con el RestClient de Spring (o JavaMail para el correo) y lee sus datos de propiedades app.*, que se llenan desde el archivo .env. Si la API falla, el sistema responde con un aviso y sigue funcionando.

Jannys

Cómo hacer funcionar las APIs, paso a paso

DNI Autocompletar nombres

1. Crea una cuenta gratis en apis.net.pe y copia tu token.
2. Abre el archivo .env en la raíz y escribe: APP_DNI_TOKEN=tu_token
3. Reinicia el servicio: .\scripts\restart-service.ps1 -Name cliente-ms
4. En «Crear cuenta», escribe un DNI y pulsa Buscar.

Sin token, la web avisa y deja escribir los datos a mano.

CORREO Mailpit local o Gmail real

Local (no hay que configurar nada)

Los correos llegan a http://localhost:28025

Real con Gmail

1. Activa la verificación en 2 pasos y crea una «clave de aplicación».
2. En .env escribe:

```
APP_MAIL_HOST=smtp.gmail.com
APP_MAIL_PORT=587
APP_MAIL_USER=tu-correo@gmail.com
APP_MAIL_PASSWORD=clave-de-aplicacion
```

3. Reinicia cliente-ms, agente-ms y reserva-ms.

Los dos

Preguntas probables del docente (1 de 2)

¿Qué evita que tu servicio quede colgado si el que llamas está caído?

Timeout (conexión 1,5 s, lectura 2 s) y Circuit Breaker: al abrirse responde rápido. En lectura devuelve datos locales con degradado = true; en escritura, 503.

¿Por qué no basta proteger rutas solo en el Gateway?

Porque se podría llamar al puerto interno de un servicio. Cada uno valida firma y emisor del JWT y que el dato sea del cliente; probado en los 9 con test-distributed-security.

¿Qué gana y qué pierde al usar eventos en vez de REST?

Gana desacople: el productor sigue aunque el consumidor esté caído. Pierde inmediatez, hay estados intermedios y hay que tratar duplicados.

¿Cómo evitas un estado inconsistente si el proceso falla a la mitad? ¿Qué es idempotente?

Saga con compensaciones persistentes y outbox/inbox. Idempotente: repetir la misma operación o evento no cambia el resultado (eventId único en inbox, X-Idempotency-Key).

Los dos

Preguntas probables del docente (2 de 2)

¿Qué evidencia del panel permite ubicar una falla sin revisar logs uno por uno?

El panel de Grafana: métricas por servicio (arriba, 5xx, p95) y logs filtrados por traceId, que muestra por qué servicios pasó la operación.

¿Por qué el frontend nunca debe conocer la dirección de una instancia?

Las instancias cambian y escalan. Eureka y el balanceador las resuelven; la web solo conoce el Gateway, así se protege y se cambia sin tocar la web.

Al migrar a Keycloak, ¿qué cambió en el Gateway y en los microservicios, y por qué la lista es corta?

Solo de dónde se validan las firmas (emisor y JWK) y el mapeo de roles desde realm_access.roles. El resto no cambia porque ya validaban un JWT estándar. El sub es String; el atributo cliente_id enlaza con el ID numérico.

CIERRE · DECISIONES PROPIAS

Lo que decidimos distinto al ejemplo del curso

Dominio propio

Inmobiliario, con 9 servicios y 3 roles.

Saga orquestada

orden-ms controla el estado y las compensaciones en orden.

Outbox e inbox

Eventos sin pérdida ni duplicados, en la misma transacción.

Pago manual verificado

El cliente sube la captura y un agente la valida.

Límites honestos: Mercado Pago real no se probó; los contratos son académicos y el pago no mueve dinero real.

Gracias · Jannys y Miguel Angel · Grupo 9